Batch: D3	**Roll No.: 16010125809**
Experiment / assignment / tutorial No. 1	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Title: : Implementation of Database in SQL -DDL

Objective: Define/modify database definitions with proper constraints

---

Expected Outcome of Experiment:
CO 2: Convert entity-relationship diagrams into relational tables, populate a relational database and formulate SQL queries on the data Use SQL for creation and query the database.

CO 3: Define and apply integrity constraints and improve database design using normalization techniques.

Pre Lab/ Prior Concepts:
Resources used: Postgresql

Theory: The set of relations in a database must be specifies to the system by means of a data definition language (DDL). The SQL DDL allows specification of not only a set of relations but also specific information about the relation including,

1. The schema for each relation
2. The domain of values associated with each attribute
3. The integrity constraints
4. The set of indices to be maintained for each relation
5. The security and authorization information for each relation
6. The physical storage structure of each relation on disk

Syntax Create Table:

create table employee(ssn,fname varchar(10), mname varchar(10), lname varchar(10), desg varchar(20), gender varchar(5), addr varchar(20), bdate datetime, sal float,primary key(ssn));

create table manages(ssn int, dept_code int, start_dt datetime, foreign key(ssn) references employee, foreign key(dept_code) references department, key(ssn,dept_code)) on delete set null

Data Constraints

Business managers of the organization determine a set of rules that must be applied before the data is stored in the database. The application of such rules on raw data ensures **data integrity**.

Eg:- An employee belonging to the Sales department cannot have a salary higher than Rs. 1000.

An employee has an unique identification number.

Applying Data Constraints

Oracle permits data constraints to be attached to table columns using SQL syntax. Constraints can be attached to table columns using Alter table.

Unique Constraint

Unique Constraint- At column level Syntax

<ColumnName><Datatype>(<size>) UNIQUE

Unique Constraint- At table level

CREATE TABLE<TableName>(

<ColumnName><Datatype>(<size>)

<ColumnName><Datatype>(<size>)

<Columnname><Datatype>(<size>) UNIQUE(<ColumnName1>,<ColumnName2>);

Implementation Details (Problem Statement, Query and Screenshots of Results):

Problem Statement –

The rapid growth of OTT platforms, managing users, subscriptions, and content has become increasingly complex. Users often face problems such as difficulty in managing multiple profiles under one account, confusion over subscription plan details, delayed or failed payments, and challenges in finding content due to unorganized catalogues. Such unstructured systems lead to data redundancy, inconsistency, and inefficient retrieval of user and content information, affecting both customer experience and business decisions.
The project proposes the design and implementation of a relational database system. The system will efficiently manage user accounts, profiles, subscription plans, payment records, and content catalogue. By enforcing relationships and constraints, it ensures data integrity, reduces redundancy, and enables fast and accurate queries.

Create Table -

```sql
create table SP_Users(
user_id int primary key,
first_name varchar(20) not null,
last_name varchar(20) not null,
email varchar(50) not null,
phone varchar(120) not null,
region varchar(20),
join_date date not null,
password varchar(50) not null,
plan_id int,
foreign key (plan_id) references SP_Subscription_Plan(plan_id)
);

create table SP_Subscription_Plan (
plan_id int primary key,
plan_name varchar(20) not null,
price int not null,
max_screens int not null,
video_quality varchar(20) not null
);

create table SP_Payment(
payment_id int primary key,
amount int not null,
payment_date date not null,
payment_method varchar(20) not null,
payment_status varchar(20) not null
);

create table SP_Content(
content_id int unique,
title varchar(20) not null,
content_type varchar(20) not null,
release_year int,
duration int not null,
age_rating varchar(20)
);
```

```sql
create table SP_Watch_History (
    watch_id int primary key,
    profile_id int not null,
    content_id int not null,
    watch_date date,
    progress_status varchar(20),
    total_watch int not null,
    foreign key (profile_id) references SP_Profiles(profile_id),
    foreign key (content_id) references SP_Content(content_id)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 23 msec.
```

```sql
create table content_type(
c_id int primary key,
content_id int,
foreign key (content_id) references SP_Content(content_id)
)
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 23 msec.
```

Select Command –

```
select * from SP_Users;
select * from SP_Subscription_Plan;
select * from SP_Content;
select * from SP_Profiles;
select * from SP_Payment;
select * from SP_Watch_History;
```

Data Output Messages Notifications

user_id [PK] integer	first_name character varying (20)	last_name character varying (20)	email character varying (50)	phone character varying (120)	region character varying (20)	join_date date	password character varying (50)	plan_id integer

Data Output Messages Notifications

plan_id [PK] integer	plan_name character varying (20)	price integer	max_screens integer	video_quality character varying (20)

Data Output Messages Notifications

content_id integer	title character varying (20)	content_type character varying (20)	release_year integer	duration integer	age_rating character varying (20)

Data Output Messages Notifications

profile_id [PK] integer	user_id integer	profile_name character varying (20)	age_limit integer

Data Output Messages Notifications

payment_id [PK] integer	amount integer	payment_date date	payment_method character varying (20)	payment_status character varying (20)

Data Output Messages Notifications

watch_id [PK] integer	profile_id integer	content_id integer	watch_date date	progress_status character varying (20)	total_watch integer

Alter Command -

```
alter table SP_Payment add column user_id int;
alter table SP_Payment drop column user_id;
```

```
alter table SP_Payment add constraint u unique (payment_id);
alter table SP_Payment drop constraint u;
```

Data Output Messages Notifications

```
ALTER TABLE

Query returned successfully in 33 msec.
```

Drop Command –

```
drop table SP_Payment;
```

Data Output | Messages | Notifications

```
DROP TABLE

Query returned successfully in 45 msec.
```

Insert Command –

```
insert into SP_Users values(1,'Mohit','Madke','mohit.madke@somaiya.edu','9999988888','Mumbai','2024-02-06','Pass123', 1);
insert into SP_Users values(2,'Samar','Solanki','samar.solanki@somaiya.edu','7777788888','Mumbai','2024-05-06','321Pass', 1);
```

Data Output | Messages | Notifications

Showing rows

	user_id [PK] integer	first_name character varying (20)	last_name character varying (20)	email character varying (50)	phone character varying (120)	region character varying (20)	join_date date	password character varying (50)	plan_id integer
1	1	Mohit	Madke	mohit.madke@somaiya.edu	9999988888	Mumbai	2024-02-06	Pass123	1
2	2	Samar	Solanki	samar.solanki@somaiya.edu	7777788888	Mumbai	2024-05-06	321Pass	1

```
insert into SP_Content values(101, 'Inception', 'Movie', 2010, 148, 'PG-13');
insert into SP_Content values(102, 'Interstellar', 'Movie', 2014, 169, 'PG-13');
```

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 31 msec.
```

```
select * from SP_Contents;
```

Data Output | Messages | Notifications

	content_id integer	title character varying (20)	content_type character varying (20)	release_year integer	duration integer	age_rating character varying (20)
1	101	Inception	Movie	2010	148	PG-13
2	102	Interstellar	Movie	2014	169	PG-13

Truncate Command –

```
132    truncate SP_Contents;
```

Data Output | Messages | Notifications

```
TRUNCATE TABLE

Query returned successfully in 32 msec.
```

Post Lab Questions:

1. **Explain in brief the following terms:**

   a. **Database**
   A database is an organized collection of structured information, or data, typically stored electronically in a computer system. It functions like a sophisticated digital filing cabinet, designed to allow users to easily store, retrieve, manage, and update large amounts of data efficiently. To facilitate this interaction, databases are usually controlled by a piece of software known as a Database Management System (DBMS), which serves as an interface between the end-user and the stored data.

   b. **Types of databases**
   - Relational Databases (RDBMS): These organize data into structured tables with defined rows and columns, making them ideal for data with clear relationships (e.g., MySQL).
   - Non-Relational (NoSQL) Databases: These are flexible systems designed to store unstructured or semi-structured data, such as documents or key-value pairs (e.g., MongoDB).
   - Cloud Databases: These run on virtual cloud platforms rather than local servers, offering easy scalability and remote accessibility (e.g., AWS RDS)

   c. **SQL Data Types**
   - Numeric Types (e.g., INT, FLOAT): These specify that a column can only hold numbers, used for calculations like quantities or prices.

- String Types (e.g., VARCHAR, TEXT): These allow for storage of alphanumeric text, such as names, addresses, or descriptions.
- Date/Time Types (e.g., DATE, DATETIME): These formats ensure that temporal data, such as birthdays or transaction timestamps, are stored consistently.

d. foreignkey

A Foreign Key is a specific field or column in one table that uniquely identifies a row in another table, effectively creating a link between the two data sets. It points to the Primary Key of a "parent" table and is used to enforce referential integrity, ensuring that the relationship between records remains valid and consistent. For example, in an "Orders" table, a "Customer ID" field would be a foreign key that connects each order back to a specific person in the "Customers" table.

2. What are the commands to:

a. Delete an entire table.

Drop table table_name;

b. To view a database.

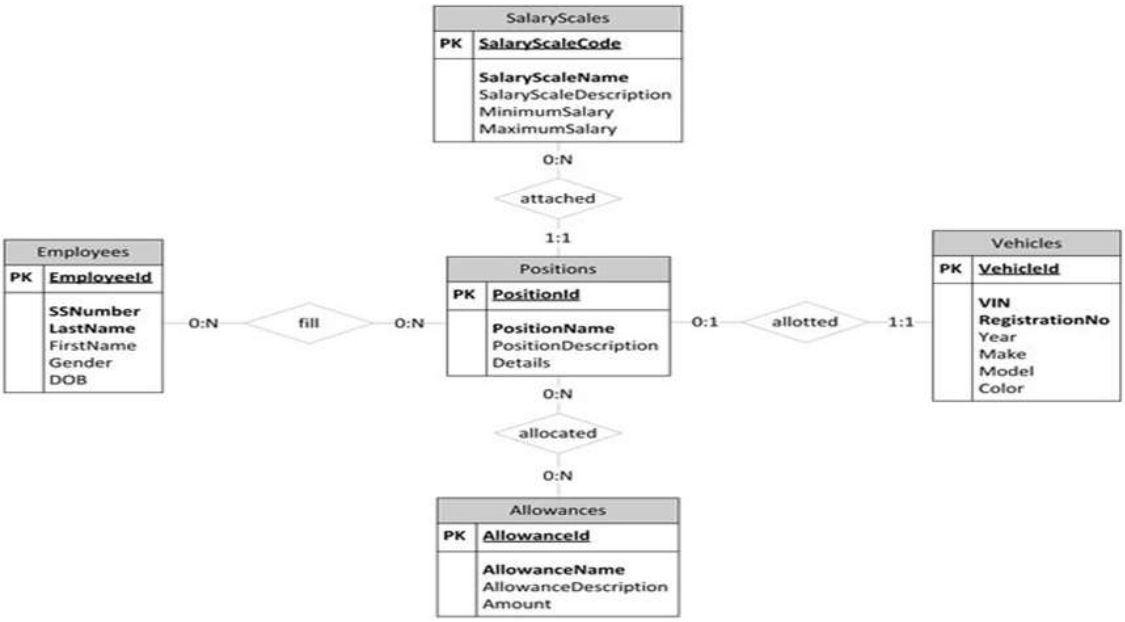
Show databases;

c. To select & view all the columns.

Select * from table_name

3. For the given ER model and relational model , using DDL commands write syntax to create CREATE Tables with all possible integrity constraints.

Problem Statement: A small accounting firm wants a simple HR application that will help it to keep track of its employees, their positions, allowances, salary scales, and which company vehicles their employees drive. The application must keep track of all the positions at the firm, the employees filling these positions, the allowances for these positions, the salary scales for these positions, and the company vehicles assigned to these positions.




```
create table salaryscales (
    salaryscalecode varchar(20) primary key,
    salaryscalename varchar(100) not null,
    salaryscaledescription varchar(255),
    minimumsalary int,
    maximumsalary int
);

create table employees (
    employeeid int primary key,
    ssnumber varchar(20) not null unique,
    lastname varchar(50) not null,
    firstname varchar(50) not null,
    gender varchar(10),
    dob date
);

create table allowances (
    allowanceid int primary key,
    allowancename varchar(100) not null,
    allowancedescription varchar(255),
    amount int
```

```
);

create table positions (
    positionid int primary key,
    positionname varchar(100) not null,
    positiondescription varchar(255),
    details varchar(255),
    salaryscalecode varchar(20),
    foreign key (salaryscalecode) references salaryscales(salaryscalecode)
);

create table vehicles (
    vehicleid int primary key,
    vin varchar(50) not null unique,
    registrationno varchar(20) not null unique,
    year int,
    make varchar(50),
    model varchar(50),
    color varchar(30),
    positionid int unique,
    foreign key (positionid) references positions(positionid)
);

create table employee_positions (
    employeeid int,
    positionid int,
    primary key (employeeid, positionid),
    foreign key (employeeid) references employees(employeeid),
    foreign key (positionid) references positions(positionid)
);

create table position_allowances (
    positionid int,
    allowanceid int,
    primary key (positionid, allowanceid),
    foreign key (positionid) references positions(positionid),
    foreign key (allowanceid) references allowances(allowanceid)
);
```

Conclusion:

Hence, after implementing the experiment, we successfully executed SQL commands such as create, alter, and drop to structure a relational database. We also learned how to manage existing table structures and remove data efficiently using truncate also gained a practical understanding of applying integrity constraints, including primary and foreign keys.